CARNalyzer

Projeto do banco de dados e integração CRUD

Documento técnico para entrega da Sprint
Data: 25 de outubro de 2026

Objetivo

Descrever a persistência de sessões de telemetria e demonstrar como o aplicativo cria, consulta, atualiza e exclui registros de forma local.

SGBD e arquitetura

O SGBD adotado é SQLite, um banco relacional embarcado no aplicativo. Não há servidor de banco remoto: os dados permanecem no dispositivo.

Android: SQLite nativo acessado pelo pacote sqflite.
Web: SQLite compilado para WebAssembly; o arquivo é persistido no IndexedDB do navegador por meio de sqflite_common_ffi_web.

A camada BancoDadosLocal isola o SQL da interface. A tela de Logs chama essa camada para todas as operações CRUD e atualiza a lista após o resultado do banco. Tema e veículo configurado continuam em SharedPreferences, pois são preferências simples, não dados de sessão.

Projeto lógico

Tabela logs

Cada linha representa uma sessão de telemetria gravada.

```
CREATE TABLE logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  vehicle TEXT NOT NULL,
  duration_seconds INTEGER NOT NULL
    CHECK (duration_seconds >= 0),
  timestamp TEXT NOT NULL,
  summary TEXT NOT NULL
);
```

Dicionário de dados

id - INTEGER; chave primária, gerada pelo SQLite.
vehicle - TEXT; identificação do veículo na sessão.
duration_seconds - INTEGER; duração total em segundos, não negativa.
timestamp - TEXT; data e hora da gravação.
summary - TEXT; resumo do estado da sessão OBD-II.

Regras e decisões

Os campos de sessão são obrigatórios. A duração recebe uma restrição
CHECK no banco e também é validada no formulário de edição.
O veículo é salvo como retrato textual da sessão; alterações futuras
na configuração atual não modificam registros históricos.
A chave id permite atualizar ou apagar exatamente uma sessão.

Integração no aplicativo

Operações CRUD implementadas

Create - ao encerrar uma gravação, inserir a sessão na tabela logs.
Read - carregar e listar sessões em ordem decrescente de inclusão.
Update - editar veículo, duração, data/hora e resumo da sessão.
Delete - remover uma sessão após confirmação explícita do usuário.

Fluxo de persistência

1. Na inicialização, abrir carnalyzer.db e criar a tabela se necessário.
2. Ler as sessões existentes e mostrá-las na tela Logs.
3. Executar INSERT, SELECT, UPDATE e DELETE com parâmetros do sqflite.
4. Atualizar a interface somente depois do resultado da operação.
5. Exibir mensagem se o banco não abrir ou uma operação falhar.

Compatibilidade e migração

Sessões da versão anterior, que estavam em SharedPreferences como JSON,
são importadas para o SQLite na primeira abertura. A preferência antiga
é removida depois da importação para evitar duplicação.
No navegador, manter a mesma origem e porta durante a demonstração:
o IndexedDB é associado à origem do site.

Arquivos principais: lib/data/banco_dados_local.dart,
lib/models/entrada_log.dart e lib/main.dart.

Demonstração em aula

Roteiro sugerido (3 a 5 minutos)

1. Executar o aplicativo em Android ou Web.
2. Abrir Telemetria e iniciar uma gravação; aguardar alguns segundos.
3. Encerrar a gravação: isso cria um novo registro no SQLite.
4. Abrir Logs e apontar a sessão recém-criada (consulta / Read).
5. Usar Editar, alterar o resumo e salvar (atualização / Update).
6. Criar uma segunda sessão e excluí-la, confirmando a ação (Delete).
7. Reiniciar o app e mostrar que o registro mantido continua salvo.

Comandos de execução e verificação

flutter pub get
flutter run -d <dispositivo>
flutter test test/banco_dados_local_test.dart
flutter build web

Para regenerar os recursos do SQLite Web após atualizar dependências:
dart run sqflite_common_ffi_web:setup
O comando cria web/sqlite3.wasm e web/sqflite_sw.js.

Evidências para apresentar

Mostrar a lista antes e depois de cada operação, a edição persistida
após reiniciar o app e o teste automatizado do ciclo CRUD no SQLite.
A configuração do veículo e o tema permanecem em SharedPreferences.